

Now, let us learn how to use SageMath on your system. We will explore three different methods, and you can choose the one you prefer. First, you can install SageMath locally on your system. Local installation of SageMath is straightforward, especially on Linux. For instance, in Ubuntu, the command `'sudo apt install sagemath'` will install SageMath. Alternatively, you can use Ubuntu Software Center or GNOME software to install SageMath directly. To open the SageMath interactive shell, execute the command `'sage'` in the terminal. This command initiates the SageMath shell, which is essentially a modified version of the IPython shell. Figure 2 shows the interactive shell of SageMath. Notice that I have SageMath version 9.0 installed in my system. In Figure 2, you can also observe a simple addition being carried out. Note that the command `'quit'` closes the SageMath shell.

The second method is to use JupyterLab, a powerful web-based interactive development environment. JupyterLab is free and open source software, and if you are not familiar with it, it may be a good idea to explore its capabilities. JupyterLab can be installed locally using the command `'pip3 install jupyterlab'`. The command `'jupyter-lab'` will launch JupyterLab in the default web browser of your system. Since you have already installed SageMath on your system, JupyterLab will automatically detect the SageMath kernel. In Figure 3, you can see the JupyterLab window, where you can select the desired programming language kernel. As I am using Python, Julia, and SageMath from JupyterLab, Figure 3 displays three kernel icons for selection. Please note that this may vary on your system. Clicking on the SageMath kernel icon (highlighted with a red box) opens a Jupyter Notebook window. These windows feature cells that can contain executable code as well as text. Notice that the extension of a Jupyter Notebook file is `'.ipynb'`.

```
# sage

SageMath version 9.0, Release Date: 2020-01-01
Using Python 3.8.10. Type "help()" for help.

sage: 111+222
333
sage: quit
Exiting Sage (CPU time 0m0.33s, Wall time 0m54.88s).
#
```

Figure 2: The interactive shell of SageMath



Figure 3: JupyterLab with SageMath kernel

```
[1]: print "Hello World"

Input In [1]
print "Hello World"
^
SyntaxError: Missing parentheses in call to 'print'. Did you m
ean print("Hello World")?

[2]: print("Hello Wold")

Hello Wold
```

Figure 4: Our first SageMath code

I have previously mentioned that the basic syntax of SageMath is similar to Python. Now, let us run our first simple SageMath code to test this statement. We will execute the following two lines of code in the Jupyter Notebook cell: `'print "Hello World"'` (Python 2) and `'print("Hello World")'` (Python 3). Figure 4 displays the Jupyter Notebook window with the output after executing the above mentioned lines of code. What observations can you make from Figure 4? Well, the Python 2 version of the code results in a syntax error, whereas the Python 3 version executes correctly. Therefore, when writing

SageMath code, please remember to consistently use Python 3 syntax, as there are subtle differences between the two versions of Python.

Now, we will discuss the third and easiest method to execute SageMath code from your system. We will use a web-based cloud computing platform called CoCalc, which directly supports SageMath worksheets. These worksheets can interactively evaluate SageMath code. The extension for CoCalc Notebooks is `'.sagews'`. An important aspect of CoCalc is that it operates on a freemium model where you have to pay for certain services.